

Java – Collections, Interfaces et Pattern Observable

Ces concepts permettent d'écrire du code plus flexible, modulaire et maintenable.

ArrayList

`ArrayList` est une collection qui permet de stocker une liste d'éléments dynamiques.

Caractéristiques :

- Taille variable
- Accès rapide par index
- Ordre conservé

```
import java.util.ArrayList;  
  
ArrayList<String> noms = new ArrayList<>();  
  
noms.add("Alice");  
noms.add("Bob");  
noms.add("Charlie");  
  
System.out.println(noms.get(0));
```

Parcourir une ArrayList

Boucle classique :

```
for (int i = 0; i < noms.size(); i++) {  
    System.out.println(noms.get(i));  
}
```

Boucle améliorée :

```
for (String nom : noms) {  
    System.out.println(nom);  
}
```

Interface

Une interface définit un **contrat** : ce qu'une classe doit faire, sans expliquer comment.

- Contient uniquement des méthodes abstraites
- Favorise le découplage
- Permet le polymorphisme

```
public interface Deplacable {  
    void deplacer();  
}
```

Implémentation d'une interface

Une classe peut implémenter une ou plusieurs interfaces.

```
public class Joueur implements Deplacable {  
    @Override  
    public void deplacer() {  
        System.out.println("Le joueur se déplace");  
    }  
}
```

Utilisation polymorphe :

```
Deplacable d = new Joueur();  
d.deplacer();
```

Pattern Observable – Principe

Le pattern Observable permet à des objets d'être **notifiés automatiquement** lorsqu'un autre objet change d'état.

- Un **Observable** : source d'événements
- Un **Observer** : écoute les changements

Objectif :

- Éviter le couplage fort
- Faciliter la communication entre objets

Interfaces du pattern Observable

```
public interface Observer {  
    void update(int valeur);  
}  
  
public interface Observable {  
    void ajouterObserver(Observer o);  
    void notifierObservers();  
}
```

Implémentation simplifiée

```
public class Compteur implements Observable {  
  
    private int valeur;  
    private ArrayList<Observer> observers = new ArrayList<>();  
  
    public void ajouterObserver(Observer o) {  
        observers.add(o);  
    }  
  
    ...  
}
```

...

```
public void incrementer() {  
    valeur++;  
    notifierObservers();  
}
```

```
public void notifierObservers() {  
    for (Observer o : observers) {  
        o.update(valeur);  
    }  
}
```

```
}
```

Observer

```
public class Affichage implements Observer {  
  
    @Override  
    public void update(int valeur) {  
        System.out.println("Valeur mise à jour : " + valeur);  
    }  
}
```

Conclusion

- `ArrayList` : gérer des collections dynamiques
- Interface : définir des contrats
- Observable : notifier des objets sans dépendances fortes

Ces concepts sont fondamentaux pour concevoir des architectures propres et évolutives.